Monte Carlo Pricer

1 Overview

Monte Carlo simulation is the most general pricing method available. Unlike the closed-form pricer — which works only when the pricing integral has an analytical solution — Monte Carlo can, in principle, price any option whose payoff can be evaluated given a simulated path. The trade-off is that it is slower and produces an estimate with statistical uncertainty rather than an exact answer.

The core idea is to exploit the risk-neutral pricing formula directly:

$$V_0 = e^{-rT}\,\mathbb{E}^{\mathbb{Q}}[\text{Payoff}(S_T)]$$

If we can simulate many draws of S_T under the risk-neutral measure $\mathbb{Q}$, we can estimate this expectation empirically by averaging the payoffs. As the number of simulated paths $N \to \infty$, the law of large numbers guarantees convergence to the true price.

2 Simulating the Terminal Stock Price

For the Black-Scholes model, simulating S_T does not require a full path — the terminal value has a known closed-form distribution. Under the risk-neutral measure, the log-return over $[0, T]$ is:

$$\ln\left(\frac{S_T}{S_0}\right) \sim \mathcal{N}\left(\left(r - q - \frac{\sigma^2}{2}\right)T,\ \sigma^2 T\right)$$

Note the drift is $r - q$ (the risk-free rate minus the dividend yield) rather than the real-world drift μ — this is the change of measure that eliminates dependence on investor preferences.

So we can sample S_T exactly as:

$$S_T = S_0 \exp\left[\left(r - q - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}\,Z\right], \quad Z \sim \mathcal{N}(0,1)$$

This is called **terminal-value sampling** (or direct simulation), as opposed to discretising the full path. For European options, where the payoff depends only on S_T, it is exact — there is no discretisation error, only the statistical noise from using finite N.

3 The Estimator

For a call option, the discounted payoff for a single simulated path i is:

$$\hat{V}_i = e^{-rT}\max(S_T^{(i)} - K,\, 0)$$

The Monte Carlo estimate is the sample mean across N paths:

$$\hat{V} = \frac{1}{N}\sum_{i=1}^{N}\hat{V}_i$$

By the central limit theorem, this estimator is approximately normally distributed around the true price V_0:

$$\hat{V} \sim \mathcal{N}\left(V_0,\ \frac{\sigma_{\hat{V}}^2}{N}\right)$$

where $\sigma_{\hat{V}}$ is the standard deviation of a single discounted payoff. The **standard error** of the estimate is $\sigma_{\hat{V}}/\sqrt{N}$, so halving the standard error requires four times as many paths. This square-root convergence is the fundamental limitation of Monte Carlo.

For an ATM call with $\sigma = 20\%$ and $T = 1$ year, the payoff standard deviation is roughly £15 per £100 notional, giving a standard error of about $15/\sqrt{N}$. With $N = 100{,}000$ paths, the standard error is ≈ 0.047 — so the estimate is typically within a few cents of the true value.

4 The `MonteCarloPricer` Class

`MonteCarloPricer` is a dataclass with two configuration fields: the number of paths and an optional random seed for reproducibility.

```
from options_pricer import (
    BlackScholesModel, EuropeanOption, MarketData, MonteCarloPricer, OptionType,
)

md    = MarketData(spot=100.0, rate=0.05)
model = BlackScholesModel(vol=0.20)
call  = EuropeanOption(option_type=OptionType.CALL, strike=100.0, expiry=1.0)

# Default: 100,000 paths, unseeded (results vary each run)
pricer = MonteCarloPricer()
print(pricer.price(call, md, model))  # ~10.45, with ~0.05 standard error

# Seeded for reproducibility
pricer_seeded = MonteCarloPricer(n_paths=200_000, seed=42)
print(pricer_seeded.price(call, md, model))  # same result every run
```

4.1 Reproducibility and Seeding

The pricer uses `numpy.random.default_rng(seed)` for random number generation. Passing `seed=None` (the default) draws a fresh seed from the OS entropy pool each time, so results will differ between runs. Passing an integer seed makes results exactly reproducible — useful for testing, debugging, or generating stable outputs in a presentation.

```
# These will produce identical results
p1 = MonteCarloPricer(n_paths=50_000, seed=7)
p2 = MonteCarloPricer(n_paths=50_000, seed=7)
assert p1.price(call, md, model) == p2.price(call, md, model)  # True
```

4.2 Path Count and Accuracy

Higher path counts reduce statistical noise. A quick rule of thumb: to achieve an accuracy of ε, you need roughly $N \approx (\sigma_{\hat{V}}/\varepsilon)^2$ paths.

```
from options_pricer import ClosedFormPricer

exact = ClosedFormPricer.price(call, md, model)

for n in [1_000, 10_000, 100_000, 1_000_000]:
    mc = MonteCarloPricer(n_paths=n, seed=42).price(call, md, model)
    print(f"N={n:>8,}  MC={mc:.4f}  error={mc - exact:+.4f}")
```

4.3 Why Monte Carlo Is Worth It

The closed-form pricer is faster and exact for vanilla European options. The Monte Carlo pricer shines when:

- The payoff depends on the **path** of S_t, not just the terminal value (barrier options, Asian options, lookbacks)
- The model has **no closed-form solution** (Heston, jump-diffusion, stochastic rates)
- You need prices for **large books** of options using the same set of simulated paths (a natural fit for GPU parallelisation)

For vanilla European options under Black-Scholes, Monte Carlo is mainly a sanity check against the closed-form result — but building it now means the same infrastructure extends immediately to more complex settings.